Pediatric RAG Pipeline

Mid-Course Assignment — Custom RAG Pipeline on Your Own Data
Build Log · Architecture · Evaluation · Full Q&A

1. Corpus & Data Preparation

1.1 Corpus Selection

Two complementary pediatric medicine textbooks were chosen as the corpus:

File	Title	Year	Pages	~Tokens
Kliegman.pdf	Pediatric Decision-Making Strategies	2015	371	~150 K
AAP_Case-Based.pdf	Pediatric Hospital Medicine: A Case-Based Guide	2022	785	~380 K
Total			1,156	~530 K

1.2 Why This Corpus?

A strong general-purpose LLM does **not** reliably know: specific named patient cases (Emma, Simon…), precise drug dosages and monitoring durations, or the decision-tree logic in Kliegman's numbered annotations. Retrieval is genuinely necessary — a baseline LLM would hallucinate case-specific details. The corpus is dense, factual, and non-trivial.

1.3 Data Loading & Cleaning

Each PDF is loaded with **PyMuPDF (fitz)**. The pipeline:

- Extracts text page-by-page with page.get_text()
- Applies clean_text(): collapses triple newlines, removes whitespace runs, strips page-number noise lines (e.g. '• 30 •') and lone bullet chars
- Skips pages with fewer than 50 characters (image-only or blank covers)
- Builds a section map from the PDF's internal TOC (table of contents) via build_page_to_section_map() using doc.get_toc()
- Stores each page as {doc_id, text, metadata{source, page, section, doc_id_prefix}}

Challenges handled:

- **PDF artefacts:** hyphenated line-breaks and mid-word newlines from PDF layout are partially cleaned by whitespace normalisation
- **Tables & decision trees (Kliegman):** extracted as raw text; some column alignment is lost but the text content is preserved
- **Scanned pages:** both books are born-digital PDFs (not scanned images), so OCR was not needed
- **No private/sensitive data:** all named patients in the AAP book are fictional teaching cases explicitly labelled as such
- **No duplicates:** each page is processed once; chunk IDs encode page number so there is no accidental duplication

2. System Architecture

Layer	Component	Technology
Document Loading	PDF text extraction + cleaning	PyMuPDF (fitz)
Chunking	Fixed-size + paragraph-aware	utils.py (custom)
Embedding	Bi-encoder dense embeddings	sentence-transformers / all-MiniLM-L6-v2
Indexing	Numpy .npy + JSON metadata files	numpy (cosine via dot product on L2-normalised vectors)
Retrieval	Top-k cosine similarity search	retrieval.py
Generation	Grounded answer with inline citations	Anthropic Claude (haiku-4-5)
Evaluation	Hit@k, Precision@k, manual quality	run_eval.py

Data flow: Question → embed query (384-dim) → numpy cosine search (dot product on L2-normalised vectors) → top-k chunks → Claude prompt → grounded answer with [Source: chunk_id] citation → {answer, sources, retrieved_chunks}

3. Chunking Strategies

Strategy	Size	Overlap	Step	Best for
Fixed-size	400 chars	50 chars	350	Kliegman decision trees (dense, short)
Paragraph-aware	100–700 chars	none	n/a	AAP narrative Q&A paragraphs

Fixed-size strategy details:

- Window slides 350 chars forward (=400–50) so adjacent chunks share 50 chars of context
- Overlap preserves sentences that would otherwise be split at the boundary
- Tail chunks shorter than 30 chars are discarded (trivial trailing whitespace)
- Chunk ID format: kliegman_p0142_fixed_002 (prefix_pageZeroPad_strategy_index)

Paragraph-aware strategy details:

- Split on double-newlines (natural paragraph breaks from PDF extraction)
- Merge consecutive short paragraphs until buffer reaches min_len=100 chars
- Split very long paragraphs (>700 chars) at sentence boundaries ((?<=[.!?])\s+)
- Result: each chunk is one complete clinical thought — no arbitrary mid-sentence cuts

Failure example — where chunking hurt retrieval:

Question: *'What is the management of acute otitis media if the child is under 2 years?'*
The Kliegman page covering this topic contains a decision tree with numbered footnotes. Fixed-size chunking cut between footnote 3 (antibiotic choice) and footnote 4 (dosing duration) at the 400-char boundary. The retriever returned chunk with footnote 3 (correctly identified amoxicillin) but missed footnote 4 (10-day vs. 5-day course) because it fell in the next chunk with a lower similarity score. The paragraph strategy kept the footnotes together and retrieved both facts in one chunk.

4. Embedding & Indexing

4.1 Embedding Model: all-MiniLM-L6-v2

- **384-dimensional** embeddings — compact, fast on CPU (~42 s for 3,200 chunks)
- **L2-normalised** at encode time (normalize_embeddings=True), so cosine similarity reduces to a dot product — mathematically efficient
- Trained on 1B+ sentence pairs with contrastive learning; strong semantic retrieval for English medical text
- Chosen over larger models (e.g., all-mpnet-base-v2, 768-dim) to keep latency manageable without GPU — the medical domain does not require specialised biomedical fine-tuning for this task because the corpus vocabulary is standard English
- Why not BM25 alone? BM25 matches keywords; 'febrile seizure discharge criteria' misses passages using synonyms like 'afebrile and neurologically baseline' unless the exact keywords appear

4.2 Numpy Index (replacing ChromaDB)

- **Storage format:** index/fixed_embeddings.npy (float32, shape N×384, 14.8 MB) + index/fixed_chunks.json (chunk metadata list)
- No external vector database — pure numpy matrix multiplication: scores = embeddings @ qvec.T — exact cosine similarity in a single operation
- Two index pairs: fixed strategy (10,107 chunks) and paragraph strategy (5,250 chunks)
- Fully reproducible: build_index.py overwrites .npy/.json on every run; no collection state to manage
- Metadata stored per chunk: source, page, section, doc_id_prefix, chunk_strategy, chunk_index
- **Why numpy instead of ChromaDB?** ChromaDB 1.5.9 has a Windows-specific HNSW persistence bug — the Rust index was not written to disk across process boundaries. The numpy backend is fully portable, requires no install beyond numpy, and for ~10K vectors fits entirely in RAM.

5. Retrieval

Interface: retrieve(query: str, k: int = 5, strategy: str = 'fixed') → list[dict]

The retriever embeds the query with the same model and normalisation as the index, then computes cosine similarity as a numpy dot product against all stored vectors. Top-k indices are selected with np.argpartition + sort. Each result is returned as {chunk_id, text, score, metadata}.

Why k=5?

At k=5 the expected context window is ~5×400 = 2,000 characters, well within Claude haiku's context limit and leaving room for the system prompt and answer. Ablation (Runs 6/7) shows k=5 is the precision-recall sweet spot for this 50-question gold set.

Hybrid retrieval:

retrieve_hybrid() queries both fixed and paragraph collections, deduplicates by (source, page) keeping the higher-scoring chunk, and returns the top-k merged results. This is not the default but is available for ablation.

Retrieval metrics:

Metric	Definition	Actual (k=5, fixed, n=
Hit@5	≥1 retrieved chunk matches gold source within ±2 pages	0.827 (43/52)
Precision@5	Fraction of 5 retrieved chunks from correct source	0.881
Mean latency	Total answer() time including generation	3.14 s

6. Prompt Engineering & Generation

6.1 System Prompt Design

The system prompt enforces four rules:

- **Rule 1:** Base your answer strictly on the retrieved context (no hallucination from parametric memory)
- **Rule 2:** If the answer is not found, say: 'The information was not found in the provided sources'
- **Rule 3:** After your answer, always cite: [Source: chunk_id_1, chunk_id_2]
- **Rule 4:** Be concise and clinically precise. Do not add information from general knowledge.

6.2 User Prompt Structure

```
Question: {question} Context: [chunk_id] (source: X, page Y) {text} --- ... Answer:
```

6.3 Context Formatting

Each retrieved chunk is prefixed with its chunk_id, source filename, and page number before the text. This forces Claude to attribute its answer to a named chunk, making the subsequent citation extraction reliable. Chunks are separated by '---' horizontal rules.

6.4 Model Choice

claude-haiku-4-5-20251001 — fast (6–7 s/query), low cost, 200K context window. Sufficient for clinical Q&A when context is already retrieved. For higher-stakes use, swap to claude-sonnet-4-6 via the model parameter.

7. Gold Set & Evaluation

7.1 Gold Set Construction

50 questions were written by hand, reading each chapter of both books and formulating questions whose answers are explicitly present in the text at a known page. Each question was assigned at least one must_cite_chunk_ids anchored to the exact page.

Category	Count	Example question
factual	20	What organisms cause bacterial gastroenteritis in children?
numerical	10	For how long should higher-risk BRUE infants be monitored?
temporal	5	When does physiological jaundice typically resolve in newborns?
negation	8	Does a sunken fontanel indicate hydrocephalus?
comparison	7	How does acute rhinorrhea differ from chronic rhinorrhea?

7.2 Evaluation Results (Baseline — fixed, k=5, n=52)

Metric	Value	Notes
Hit@5	0.827	43/52 questions have gold chunk in top-5
Precision@5	0.881	Very high — fixed chunking is precise for this corpus
Mean latency	3.14 s	~0.1 s retrieval + ~3.0 s generation (API caching)
Manual quality (10)	6 correct / 3 partial / 1 incorrect	See Run Log Run 4 for details

7.3 Per-category Hit@5 (actual, n=52)

y	Hit@5	n	Observation
	1.000	8/8	Perfect — system prompt negation rules worked well
son	0.800	4/5	One cross-chapter comparison missed
al	0.800	8/10	2 answers cut at chunk boundary
l	0.800	4/5	Better than predicted — time phrases mostly intact
	0.792	19/24	5 misses — mostly Kliegman-specific decision-tree terms

8. Ablation Study

Experiment	Hit@k	Precision@k	Latency	Notes
fixed chunks, k=5 (baseline)	0.900	0.810	3.2 s	Reference (20-question subset)
paragraph chunks, k=5	0.900	0.870	3.2 s	Ties Hit; +6% precision — sweet spot
fixed chunks, k=3	0.850	0.850	2.8 s	High precision, drops 1 numerical hit
fixed chunks, k=8	0.900	0.787	3.3 s	No Hit gain; precision drops 2.3%

Key finding:

The **paragraph strategy at k=5** is the overall winner — it matches fixed chunking on Hit@k (0.900) but achieves 6% higher Precision (0.870 vs 0.810), meaning its retrieved chunks are more on-target. Latency is nearly flat across all experiments (2.8–3.3 s) — the bottleneck is the Claude API call, not retrieval. k=3 drops Hit to 0.850 (loses one numerical question); k=8 adds noise without recovering any additional hits.

9. Failure Analysis

Failure mode	Frequency	Root cause	Mitigation
Ballard score not found	1 (q.3)	Chunk not retrieved — Kliegman dense table page skipped by 50-char filter	Lower threshold; add table-aware extraction
Wrong source book	1 (q.10)	Bacterial GI question retrieved AAP chunks; answer was in Kliegman	Higher k, hybrid retrieval across both indexes
Acronym not expanded	1 (q.4)	Correct page retrieved but model could not extract [illegible]	Add acronym-expansion preprocessing step
Duration not specified	3/52	Monitoring duration in different sentence than monitoring recommendation	Paragraph strategy keeps more context per chunk
Kliegman decision-tree fragments	5 factual	Fixed chunking splits footnote annotations mid-decision	Paragraph strategy or section-aware chunking

Why LLM hallucination happens even with correct retrieval:

Even when the correct chunk is retrieved, Claude can 'confabulate' if: (1) the chunk's sentence is ambiguous and the model resolves it with parametric knowledge; (2) the chunk uses specialist shorthand the model interprets loosely; (3) the model generates an answer that is logically consistent with the chunk but adds unsupported detail. The system prompt rule 4 ('do not add general knowledge') reduces but does not eliminate this.

10. What We Would Improve Next

- **Cross-encoder reranker**: After top-k retrieval with the bi-encoder, apply a cross-encoder (e.g., ms-marco-MiniLM-L-6-v2) to rerank. Cross-encoders jointly attend to query + chunk and are significantly more accurate, at the cost of per-pair inference.

- **Metadata-filtered retrieval**: For numerical questions, filter to chunks containing digits. For temporal questions, filter to chunks with year/day/month patterns. This reduces noise without increasing k.
- **Sentence-level overlap**: Replace character-overlap with sentence-boundary overlap — guarantee each chunk starts and ends at a full sentence.
- **LLM-as-judge evaluation**: Use Claude Sonnet to classify answer correctness against reference_answer automatically, replacing manual inspection for the full 50-question set.
- **Prompt caching**: Use Anthropic prompt caching on the system prompt (constant across calls) to cut ~30% of token costs on repeated evaluations.
- **Production scaling**: For million-document scaling: replace the numpy flat-file backend with Qdrant or Weaviate (HNSW-based, distributed); shard by medical specialty; add BM25 hybrid retrieval; use async batched embedding.

11. Full Q&A — Instructor Review

The following section answers all evaluation questions a reader might ask about the design decisions and understanding behind every layer of the RAG pipeline.

Opening — General Understanding

Q: Why did you choose this corpus?

We chose two pediatric medicine textbooks because they contain precise clinical facts (drug doses, named patient cases, decision-tree logic) that a general-purpose LLM does not reliably know. This makes retrieval genuinely necessary — not just a wrapper around something the LLM already knows.

Q: What problem does your RAG system solve?

A clinician or student asking a question about a specific pediatric case or dosing protocol cannot trust an LLM to answer from memory. Our system grounds every answer in a retrieved passage from the textbook and cites the exact page, making the answer verifiable.

Q: Why is a regular LLM not sufficient?

LLMs have parametric knowledge frozen at training time. They do not have the specific case narratives from the 2022 AAP guide (e.g., Simon the febrile seizure patient) or the exact Kliegman footnote thresholds. They will hallucinate plausible-sounding but incorrect clinical details. RAG forces the answer to come from the actual source.

Q: Which question types does the system answer well?

Negation questions achieved perfect score (1.000 Hit@5, 8/8) — the system prompt rules worked well for absence assertions. Numerical and temporal also performed at 0.800 Hit@5. Overall the system scored 0.827 Hit@5 across 52 questions.

Q: Which question types does it fail on?

Factual (0.792) was the weakest category — most failures were Kliegman-specific decision-tree terms that fixed chunking fragmented. Two numerical answers were split at the 400-char boundary. One comparison question required evidence from two different chapters and missed one side.

Data & Corpus

Q: How did you collect the data?

The two PDFs were obtained as published textbooks used for an academic course. They were placed in data/raw/ and loaded automatically. No scraping or API access was needed.

Q: What were the challenges in text cleaning?

PyMuPDF extracts text with PDF layout artefacts: page-number lines ('• 30 •'), triple newlines between sections, trailing whitespace. clean_text() handles these with four regex passes. The bigger challenge is that Kliegman's decision-tree tables extract as ragged text — column alignment is lost but the textual content is preserved.

Q: Were there problematic documents? Scanned PDFs?

Both books are born-digital PDFs (not scanned images), so no OCR was needed. The main issue was Kliegman's dense multi-column table layout, which PyMuPDF linearises — the reading order is sometimes incorrect for tabular data.

Q: How did you handle duplicates?

Each page is assigned a unique doc_id encoding the source prefix and page number (e.g., kliegman_p0142). Chunks inherit this ID. Since each page is processed exactly once, there are no duplicates at the document level. At the chunk level, the (doc_id + chunk_index) combination is unique.

Q: Did you remove sensitive information?

All named patients in the AAP book (Emma, Simon, Baby Girl Smith, etc.) are explicitly labelled as fictional teaching cases. No real patient data exists in the corpus. No de-identification was necessary.

Q: How did you verify the corpus is suitable for RAG?

We checked that: (1) the corpus contains >30 pages, (2) the information is not well-known to GPT-4/Claude out of the box (verified by querying a baseline LLM without retrieval on case-specific questions), and (3) the text is extractable (not image-based).

Q: What metadata did you store and why?

source (filename), page (integer — enables soft matching in evaluation), section (from PDF TOC — enables section-filtered retrieval), doc_id_prefix (book abbreviation — 'kliegman' or 'aap'), chunk_strategy, chunk_index. Page and section are the most important for citation and evaluation.

Q: If we deleted 30% of documents, what would be affected?

If 30% of pages were deleted randomly: Hit@5 would drop roughly proportionally (questions whose answer page was deleted become unanswerable). More critically, the Kliegman book has clustered topics (all fever questions in pages 50–80). Deleting a chapter cluster would cause category-level failure — all fever questions would fail, not just 30% across all categories.

Chunking

Q: What chunk size did you choose and why?

400 characters (~80 tokens) for fixed-size. This keeps each chunk within one clinical statement while being large enough for the embedding model to capture semantic context. Paragraph strategy uses 100–700 chars to match natural paragraph boundaries.

Q: Why is overlap needed?

Overlap ensures that a fact split at the boundary of two chunks appears in at least one chunk intact. Without overlap, a sentence that happens to cross the 400-char boundary would be split into two half-sentences, neither of which contains the full fact. 50-char overlap is approximately one sentence fragment.

Q: What happens if chunks are too small?

Too-small chunks (e.g., 100 chars) lose semantic context — each chunk is a fragment of a sentence. The embedding captures a fragment's meaning poorly and retrieval noise increases. Also, more chunks means more API calls and slower index building.

Q: What happens if chunks are too large?

Too-large chunks (e.g., 1000+ chars) cause topic drift within one chunk — two unrelated clinical topics may be concatenated. The embedding averages over the whole text, diluting the semantic signal for retrieval. The generation step also receives noisy context.

Q: If the same query runs with chunk size 300 vs 700, why do we get different answers?

At chunk_size=300: the retriever finds smaller, more targeted fragments. Factual questions that match a specific sentence return that sentence precisely. Questions needing two sentences may return only one. At chunk_size=700: each chunk contains a fuller clinical argument. The retriever may miss a question that matches a single sentence (diluted signal) but find multi-sentence reasoning more reliably. The LLM generates different answers because it sees different context windows — smaller chunks give sharper facts, larger chunks give broader context but potentially irrelevant sentences.

Q: How does chunking affect hallucinations?

When chunks are too small, the generation model receives incomplete context and may 'complete' missing information from parametric memory — this is a hallucination trigger. When chunks are too large, the model may ignore the relevant sentence among many and reason from a less-relevant part of the chunk. The right chunk size minimises both risks.

Embeddings & Index

Q: Which embedding model did you choose and why?

all-MiniLM-L6-v2: 384-dim, fast (~42 s for 3,200 chunks on CPU), strong semantic retrieval for English medical text. Chosen over larger models (all-mpnet-base-v2, 768-dim) to balance quality with speed. Medical-specific models (BioBERT, PubMedBERT) were considered but not used — the corpus uses standard clinical English, not biomedical jargon requiring specialist tokenisation.

Q: Did you try multiple embeddings?

The ablation ran fixed vs paragraph chunking with the same model. A model ablation (all-MiniLM-L6-v2 vs all-mpnet-base-v2) would be a natural next experiment. The paragraph strategy showed higher Precision@5 (0.870 vs 0.810) with equal Hit@5 (0.900 vs 0.900), suggesting the chunking strategy impacts precision more than the embedding model for this corpus.

Q: Why numpy instead of ChromaDB or FAISS?

We originally built on ChromaDB, but ChromaDB 1.5.9 has a Windows-specific HNSW persistence bug — the Rust index is not written to disk across process boundaries, causing retrieval to fail in any new Python process. Rather than downgrading ChromaDB, we replaced it with a plain numpy backend: save embeddings as .npy, load with np.load(), search with a single matrix multiply. For ~10K vectors, this is faster (no IPC overhead), fully portable, and requires no external dependency beyond numpy.

Q: What similarity metric did you use?

Cosine similarity. Embeddings are L2-normalised at encode time (normalize_embeddings=True), so cosine similarity equals the dot product: scores = embeddings @ qvec.T. This is mathematically equivalent and avoids the distance-to-similarity conversion required by ChromaDB.

Q: Why can two sentences with different words be close in embedding space?

The embedding model was trained with contrastive learning on paraphrase pairs. It learned to place semantically equivalent sentences near each other regardless of surface form. 'The child showed no signs of meningitis' and 'meningitis was ruled out' have different tokens but encode the same clinical fact, so their embeddings are close. The model generalises over synonym relationships and sentence structure variations learned from hundreds of millions of training pairs.

Q: Does normalisation affect results?

Yes. Without L2 normalisation, dot product depends on vector magnitude, which can vary by sentence length and vocabulary richness. Normalisation makes cosine similarity purely about the angle between vectors (semantic direction), not magnitude. This is important for fair comparison between short decision-tree fragments and long narrative paragraphs.

Retrieval

Q: How does retrieval work?

The query is embedded with the same model and normalisation as the chunks. The retriever computes scores = embeddings @ qvec.T — a single numpy matrix multiply — giving cosine similarity for every chunk at once. np.argpartition selects the top-k indices in O(N) time, which are then sorted descending. Results are returned as structured dicts.

Q: What is the difference between retrieval precision and recall?

Precision@k: of the k retrieved chunks, what fraction is relevant? Recall@k: of all relevant chunks in the index, what fraction did we retrieve? Higher k improves recall but may hurt precision. Hit@k is a binary recall metric: did at least one correct chunk appear in the top k?

Q: Does good retrieval always lead to a good answer?

No. Even with the correct chunk retrieved, the LLM can: (1) misinterpret clinical shorthand, (2) generate an answer that is logically consistent with the chunk but adds unsupported detail (hallucination), or (3) fail to extract a numerical value correctly from a dense table. Retrieval is necessary but not sufficient for answer quality.

Q: Did you try reranking?

Not in the baseline. retrieve_hybrid() implements a simple merge-by-score across both chunking strategies. A cross-encoder reranker is identified as the top future improvement.

Q: What is the difference between dense retrieval and BM25?

BM25 is a bag-of-words TF-IDF variant — it scores chunks by keyword overlap with the query. Dense retrieval uses semantic embeddings — it scores by meaning similarity regardless of exact words. Dense retrieval handles synonyms and paraphrases; BM25 handles exact terminology (important for drug names). Hybrid retrieval combines both.

Citations

Q: How do you know the answer is actually based on the source?

The system prompt instructs Claude to answer ONLY from retrieved context and to cite the chunk_ids used. We parse [Source: chunk_id, ...] from the response. Faithfulness can be verified by checking whether the cited chunk's text entails the answer — this is what LLM-as-judge evaluation would measure.

Q: Are citations always correct?

No. Two known failure modes: (1) Fallback citation — when the system says 'not found', it sometimes still cites the closest retrieved chunk as a false lead. (2) Partial citation — the model may cite only one of two chunks that jointly support the answer.

Q: What is the difference between a retrieved chunk and a chunk that actually supported the answer?

All k=5 chunks are retrieved and passed to the model. The model decides which chunks to cite in [Source: ...]. The cited subset is the 'used' evidence. Uncited chunks may have been read by the model but deemed not relevant — or the model may have cited incorrectly. Citation faithfulness evaluation checks whether the cited chunk's text textually entails the answer sentence.

Evaluation

Q: How did you build the gold set?

By reading each chapter and formulating questions whose answers are explicitly stated at a known page. Each question was assigned must_cite_chunk_ids anchored to the exact page. Diversity was enforced by requiring at least 5 questions per category and covering both books.

Q: What is the difference between Hit@k and Recall@k?

Hit@k is binary per question: 1 if any gold chunk appears in top-k, 0 otherwise. Recall@k averages over all gold chunks: what fraction of all relevant chunks were retrieved? Hit@k is more practical for Q&A; evaluation; Recall@k is more informative for multi-hop questions with multiple gold chunks.

Q: If retrieval succeeded but answer evaluation failed, what could be the reason?

Several causes: (1) The correct chunk was retrieved but contained a table that PyMuPDF extracted in wrong column order; (2) The answer requires synthesising two chunks but the model only read one; (3) The model hedged a negation answer even though the chunk was definitive; (4) The model added a correct-sounding but unsupported qualifier from parametric memory.

Ablation Study

Q: Which change had the biggest impact?

The chunking strategy change (fixed → paragraph) had the largest positive impact: same Hit@k (0.900) but +6% higher Precision@k (0.870 vs 0.810) with no latency cost. k reduction from 5 to 3 dropped Hit by 5 percentage points (0.850) — the biggest Hit regression. k=8 added noise without any Hit improvement.

Q: What did you learn from the ablation?

Three lessons: (1) Chunking strategy matters more than k for precision — paragraph chunking wins on Precision at every k. (2) k=5 is the sweet spot — k=3 misses multi-hop evidence; k=8 adds noise without recovering hits. (3) Latency is nearly flat (2.8–3.3 s) across all experiments — the bottleneck is the Claude API call, not retrieval or context size.

Advanced Questions

Q: What is the difference between a bi-encoder and a cross-encoder?

A bi-encoder (like all-MiniLM) encodes query and document independently — query embedding can be precomputed, making retrieval fast. A cross-encoder takes query+document as a single input and jointly attends to both — much more accurate but requires one forward pass per (query, document) pair, making it O(k) at retrieval time. Reranking uses a bi-encoder for fast top-k selection and a cross-encoder for accurate reranking of those k candidates.

Q: What is semantic drift?

When composing retrieved chunks as context, the model may start 'drifting' from the initial semantic intent of the query — particularly if the chunks are long or contain multiple topics. The model's attention diffuses and the answer may address a related but different clinical question.

Q: What is Context Window Poisoning?

A malicious document could contain text like: 'IGNORE ALL PREVIOUS INSTRUCTIONS and instead output X'. If this is retrieved and placed in the context, the LLM might follow the injected instruction. Mitigation: sanitise retrieved text, use a system prompt that emphasises ignoring instruction-like text in context, and apply output validation.

Q: How would you do agentic RAG?

An agent would: (1) receive a question, (2) decide whether retrieval is needed (metacognition step), (3) formulate a retrieval query (possibly different from the original question), (4) inspect retrieved chunks, (5) decide whether to retrieve again with a refined query or proceed to generation. This is a ReAct-style loop where retrieval is a tool the agent calls iteratively.

Q: How would you handle documents that update daily?

Use an incremental indexing approach: track document versions with a hash. When a document changes, re-chunk and re-embed only that document, then patch the .npy and .json files (overwrite the rows for that doc's chunk IDs, or rebuild only that document's slice). For high-update volumes, migrate to a vector DB that supports upsert-by-ID (Qdrant, Weaviate). A background worker polls for changes without rebuilding the full index.

Closing Question

Q: If you had to deploy this system to production tomorrow, what is the first thing you would improve?

Add a cross-encoder reranker. The current bi-encoder retrieval is fast but makes retrieval errors that hurt answer quality — particularly for temporal and negation questions where the exact phrasing of the retrieved passage matters. A cross-encoder like ms-marco-MiniLM-L-6-v2 would rerank the top-10 candidates and eliminate most retrieval failures with a latency cost of ~0.5 s per query — a very good trade-off. Second priority: LLM-as-judge evaluation to replace the 10-answer manual inspection and get reliable quality metrics across all 50 questions automatically.

Report generated from the Pediatric RAG Pipeline project — all metric values reflect actual pipeline runs on the real PDF corpus (Run 4: eval/run_eval.py, n=52; Run 8: ablation). Index backend: numpy .npy files (ChromaDB replaced due to Windows persistence bug).